



# Урок 9 Дополнительные возможности SQL

## Урок 9 Дополнительные возможности SQL

### Представления

Когда у нас есть несколько таблиц-источников, которые мы объединяем, фильтруем и агрегируем, результат такого запроса может использоваться многократно, а многократное написание запроса может стать трудозатратным. В этом случае можно создать представление (или `VIEW`) на основе этого запроса, то есть фактически дать запросу свое имя. Затем мы можем обращаться к представлению так, как будто это таблица. Но на самом деле представление не является таблицей, а данные хранятся в исходных таблицах-источниках. Каждый раз при обращении к представлению выполняется тот же самый запрос: объединение, фильтрация, агрегация данных. Таким образом, ресурсы не экономятся, но это удобный способ сделать запрос более читаемым или предоставить заказчикам удобный вид результата.

Несмотря на то, что представления, как и CTE используются в схожих целях, между ними есть различия:

- Представление можно использовать в других запросах без повторного его определения (в отличие от CTE, который живет в рамках одного запроса). Это объясняется тем, что представление является физическим объектом в

базе данных и хранится на диске (не забываем, что хранится только запрос, а не данные, возвращаемые этим запросом).

- CTE может быть частью представления. Это помогает, например, когда нам необходимо выполнить рекурсивный запрос, который не возможен без использования CTE.

### Пример создания представления:

```
1 CREATE VIEW only_human AS
2 SELECT id, name, status, type, gender, origin_id
3 FROM characters
4 WHERE species = 'Human';
```

Мы создаем представление с помощью инструкции `CREATE VIEW`, указываем название представления, и затем добавляем `SELECT`-запрос, который хотим именовать. Теперь мы можем обращаться к этому представлению как к таблице согласно синтаксису SQL:

```
1 SELECT * FROM only_human;
```

### Преимущества:

- Упрощение работы с часто используемыми запросами, так как с представлением можно работать в разных запросах
- Улучшение читаемости и удобства при предоставлении данных заказчикам
- Представление может использоваться для ограничения доступа определенных пользователей к базе данных (мы можем предоставить пользователям доступ к определенным представлениям, которые запрашивают данные, которые им разрешено видеть, не раскрывая всю базу данных).



Данные в представлении не хранятся. Каждый раз при обращении к представлению исходные таблицы объединяются, фильтруются и агрегируются заново. Таким образом, при использовании представления не происходит экономии вычислительных ресурсов.

## Материализованные представления

Материализованные представления выглядят как обычные, но результаты запроса сохраняются в физическую таблицу (которая хранится на диске). Они поддерживаются не во всех СУБД, а работа с ними может существенно различаться. Например, в некоторых системах данные обновляются в реальном времени, в других — по расписанию или по триггеру, а в некоторых такого функционала вообще нет. Поэтому, работая с материализованными представлениями, всегда нужно обращаться к документации, чтобы понять, как они реализованы в конкретной СУБД.

## Оптимизация запросов

Когда мы пишем запрос, внутри движка СУБД он попадает в оптимизатор. Оптимизатор стремится сделать так, чтобы запрос выполнялся быстрее и с минимальными затратами ресурсов. Он анализирует статистику таблиц, выбирает наиболее подходящие индексы и иногда переписывает запрос, чтобы улучшить его работу. Уровень «умности» оптимизатора зависит от конкретной СУБД: в одних случаях он более продвинут, в других менее, а в некоторых мы можем подсказать оптимизатору, какой индекс использовать.

Оптимальные запросы важны по нескольким причинам:

1. **Производительность:** в production-системах каждая миллисекунда важна, чтобы данные быстро возвращались пользователю.
2. **Совместное использование ресурсов:** в аналитических базах, где работает много пользователей, необходимо минимизировать использование ресурсов, чтобы не мешать другим пользователям.
3. **Экономия:** в облачных базах данных вы в том числе платите за использованные вычислительные ресурсы, поэтому оптимизация запросов позволяет сэкономить деньги.

Есть множество способов оптимизации запросов, но они различаются в разных СУБД. Однако есть несколько универсальных подходов, которые мы затронем.

## Основные способы оптимизации

### Индексы

Для лучшего понимания можно выстроить аналогию. Представим библиотеку, где нужно найти конкретную книгу. Перебор полок займет много времени, поэтому мы обращаемся к библиотекарю, который использует картотеку. В

одной картотеке книги отсортированы по фамилии автора, в другой — по названию, в третьей — по году издания. Напротив каждого названия указана полка, где можно найти нужную книгу. Индексы работают аналогично картотеке: они ускоряют поиск данных, но требуют хранения информации и обновления при изменениях.

Создание индексов на всех полях таблицы нецелесообразно. Во-первых, это увеличивает занимаемое место на диске. Во-вторых, добавление, изменение или удаление записей приводит к необходимости обновления индексов. Лучше создавать индексы на тех полях, которые часто используются в фильтрации или объединении таблиц. Оптимизатор использует индексы, чтобы быстрее вернуть данные. В разных СУБД индексы работают по-разному, и их использование зависит от статистики, подсказок пользователя в запросах и других факторов.

## Анализ плана запроса

Анализ плана запроса позволяет понять, как именно данные собираются, стоимость (cost) запроса, какие индексы и соединения ( `JOIN` ) используются. Для этого в запрос добавляют ключевое слово `EXPLAIN` :

```
▼
1 EXPLAIN
2 SELECT ...      -- запрос
```

## Примеры оптимизации

Пример — фильтрация по дате:

```
▼
1 SELECT *
2 FROM big_table
3 WHERE year(date) = '2021';
```

Здесь используется функция `year()` , из-за которой индекс на поле `date` не работает. Индексы зависят от конкретного поля, и функции могут их блокировать. Лучше использовать конструкцию `BETWEEN` :

```
▼
1 SELECT *
2 FROM big_table
```

```
3 WHERE date BETWEEN DATE '2021-01-01' AND DATE '2021-12-31';
```

Этот запрос менее читаемый, но более оптимальный.

### Еще один пример — оптимизация соединений:

```
1 SELECT *  
2 FROM first_table AS f  
3 JOIN second_table AS s  
4 ON s.id = f.id;
```

Если нужно фильтровать данные из второй таблицы, например, по условию соответствия года, лучше добавить условие в **ON**, чтобы избежать лишней фильтрации после соединения:

```
1 SELECT *  
2 FROM first_table AS f  
3 JOIN second_table AS s  
4 ON s.id = f.id AND s.year = '2021';
```

## Регулярные выражения

Регулярные выражения позволяют находить строки, соответствующие заданным условиям. Они сложны для чтения, но полезны.

### Примеры использования в SQL

- Поиск строк с регистрозависимым совпадением:

```
1 SELECT *  
2 FROM employees  
3 WHERE name ~ '[0-9]';
```

- Регистронезависимое совпадение:

```
1 SELECT *  
2 FROM employees
```

```
3 WHERE name ~* 'alice';
```

- Замена вхождений:

```
1 SELECT regexp_replace(name, 'Ваня', 'Иван', 'g')
2 FROM employees;
```

Флаг `'g'` указывает на замену всех вхождений.

- Извлечение совпадений:

```
1 SELECT regexp_matches(name, '\b[Aa]\w+', 'g')
2 FROM employees;
```

В разных СУБД регулярные выражения работают немного по-разному, но основной принцип одинаков — поиск, замена и извлечение подстрок.

Пример регулярного выражения, проверяющий соответствие пароля указанным в выражении правилам:

```
1 ^(?=[A-Z])(?=[a-z])(?=[\d])(?=[@!%*?&])[A-Za-z\d@!%*?&]{8,}$
```

В данном выражении проверяется, что во введённом пароле присутствует:

- Как минимум одна заглавная буква `(?=[A-Z])`
- Как минимум одна строчная буква `(?=[a-z])`
- Как минимум одна цифра `(?=[\d])`
- Как минимум один специальный символ `(?=[@!%*?&])`
- Не менее восьми символов `[A-Za-z\d@!%*?&]{8,}`

Тема регулярных выражений стоит немного в стороне, но при этом стоит того, чтобы в нее погрузиться. Предлагаем ознакомиться на [Хабре](#) со статьей от коллег из Росбанка.

## Рекурсивные запросы

Рекурсия — это вызов функции самой себя. В SQL рекурсивные запросы позволяют собирать данные в цикле. Они полезны для построения иерархий, таких как организационные структуры или получение упорядоченных списков (например, всех дат за период).

### Пример рекурсивного запроса в PostgreSQL:

```
1 WITH RECURSIVE numbers AS (  
2   SELECT 1 AS num  
3   UNION ALL  
4   SELECT num + 1  
5   FROM numbers  
6   WHERE num < 20  
7 )  
8 SELECT num FROM numbers;
```

Этот запрос создает список чисел от 1 до 20. Начальное значение — 1, затем в цикле добавляется 1, пока не достигнем 20.

### Другой пример — получение всех дат за определенный период:

```
1 WITH RECURSIVE dates AS (  
2   SELECT '2027-01-01'::DATE AS date  
3   UNION ALL  
4   SELECT date + INTERVAL '1 day'  
5   FROM dates  
6   WHERE date < '2027-03-31'::DATE  
7 )  
8 SELECT date FROM dates;
```

Этот запрос возвращает все даты с 1 января по 31 марта 2027 года.

## Вопросы для собеседования

### 1. Зачем используются представления?

Представления упрощают сбор данных из одних и тех же таблиц-источников, уменьшая количество одинакового кода.

### 2. Чем отличается материализованное представление от обычного?

Материализованное представление — это физическая таблица, которая обновляется по определённому событию, в отличие от обычного

представления, которое является просто именованным запросом.

### 3. Что произойдёт, если изменится схема таблицы, на основе которой создано представление?

Представление сломается, и его нужно будет пересоздать, используя `DROP VIEW` и `CREATE OR REPLACE VIEW`.

## Подготовка к собеседованию

### Паттерны решения задач

В этом уроке разберем основные паттерны написания SQL-запросов, которые пригодятся как на собеседованиях, так и в работе.

#### Количество записей по дням

Первый паттерн позволяет определить, сколько записей поступило к нам в каждый из дней. Для этого используем следующий запрос:

```
1 SELECT date,  
2        COUNT(1)  
3 FROM orders  
4 GROUP BY date  
5 ORDER BY date DESC;
```

Этот паттерн позволяет заметить критическое падение количества записей в таблице. Мы группируем данные по дням, подсчитываем количество записей за каждый день и сортируем результат в обратном порядке, чтобы актуальные даты отображались первыми. Это помогает визуально определить дни, когда произошло уменьшение записей, и понять причины: может, к нам пришло больше клиентов или случился отток, или возникла проблема с интеграцией. Таким образом, мы сможем точнее определить, в какой день это произошло.

#### Обнаружение отсутствующих дат в последовательности

Более сложным вариантом паттерна выше является обнаружение дыр в последовательности.



```

1 WITH RECURSIVE dates AS (
2     SELECT '2027-01-01'::DATE AS date
3     UNION ALL
4     SELECT date + INTERVAL '1 day'
5     FROM dates
6     WHERE date < '2027-03-31'
7 )
8 SELECT o.date,
9        COUNT(1)
10 FROM dates
11 LEFT JOIN orders AS o
12     ON dates.date = o.date
13 GROUP BY o.date
14 ORDER BY date DESC;

```

В этом запросе мы рекурсивно создаем список всех дат и накладываем на него таблицу с записями по конкретным дням. Разница с предыдущим запросом в том, что здесь мы увидим не только падение или увеличение количества строк, но и отсутствие записей за определенные дни. Это позволяет определить даты, в которые нет записей.

Посмотрим на наших данных, как это можно сделать.

Возьмем таблицу эпизодов. Если бы эпизоды выходили каждый день или по несколько раз в день, запрос мог бы выглядеть так:

```

1 SELECT date_trunc('month', air_date)::date AS month, COUNT(1)
2 FROM episodes
3 GROUP BY date_trunc('month', air_date)
4 ORDER BY 1 DESC;

```

Этот запрос показывает все даты. Обратите внимание на форматирование даты — Redash интерпретирует дату как 01/09/21, хотя хранится она как 2021-09-01. Здесь мы видим, отсортированные в обратном порядке, месяцы и количество эпизодов, вышедших в каждый месяц.

Теперь создадим рекурсивный запрос:

```

1 WITH RECURSIVE date_series AS (
2     SELECT DATE '2013-12-01' AS first_day
3     UNION ALL
4     SELECT (first_day + INTERVAL '1 month')::date
5     FROM date_series

```

```
6 WHERE first_day < DATE '2021-09-01'
7 )
8 SELECT first_day, COUNT(e.id)
9 FROM date_series
10 LEFT JOIN episodes AS e
11 ON first_day = date_trunc('month', air_date)::date
12 GROUP BY first_day
13 ORDER BY first_day DESC;
```

Первая дата выхода сериала — декабрь 2013 года, поэтому мы начинаем с 2013-12-01 и приводим к типу `date` для удобства. Затем мы добавляем к дате один месяц и выбираем дату последнего эпизода в базе, это сентябрь 2021 года. Таким образом, мы получаем список дат. Далее мы присоединяем к этим датам таблицу эпизодов с помощью `LEFT JOIN`, чтобы не потерять ни одной даты. Приведение даты эпизода к типу `date` возьмем из предыдущего запроса. После этого мы увидим не только месяцы, в которые выходили эпизоды, но и месяцы, в которые не было ни одного эпизода, что позволяет выявить дыры в данных.

## Количество типов записей в поле

Следующий паттерн пригодится, если нам нужно одним запросом выяснить, сколько записей разных типов содержится в таблице. Например, у нас есть таблица пользователей, и мы хотим узнать количество пользователей с базовой и расширенной подпиской.

В запросе ниже мы хотим узнать, сколько мужчин и женщин воспользовались вашим продуктом:

```
1 SELECT COUNT(1) AS all_cnt,
2       SUM(CASE WHEN sex = 'male' THEN 1 ELSE 0 END) AS male_cnt,
3       SUM(CASE WHEN sex = 'female' THEN 1 ELSE 0 END) AS female_cnt
4 FROM people;
```

В первой строке мы возвращаем общее количество записей в таблице, чтобы проверить наличие пустых полей или полей с другими значениями. Далее мы используем конструкцию `CASE`: если пол пользователя равен `male`, мы ставим единицу, иначе — ноль; в третьем поле делаем наоборот. В итоге суммируем количество единиц. Так мы получаем общее количество пользователей, мужчин и женщин одним запросом, без `UNION` и группировок.

## Антиджоин

Следующий паттерн часто называют антиджойн. При использовании левого или правого джойна мы соединяем одну таблицу с другой, но иногда нас интересуют строки, для которых нет соответствий во второй таблице. Это может понадобиться, например, для получения списка клиентов, которые не сделали ни одной покупки. В таком случае мы используем **LEFT JOIN** и исключаем строки, имеющие соответствия во второй таблице.

```
1 SELECT a.*
2   FROM table_a AS a
3  LEFT JOIN table_b AS b
4    ON a.id = b.id
5 WHERE b.id IS NULL;
```

Это и называют антиджойн. Его логику можно реализовать через **NOT EXISTS**. Хотя антиджойн не является официальным типом джойна, вы можете упомянуть его на собеседовании, если понадобится.

## Дубли в таблице

Следующий паттерн позволяет найти дубли в таблице. Например, вы даете большую скидку на первую покупку, пользователь регистрируется и получает скидку. Но вдруг вы забыли сделать проверку на существующий e-mail в базе. Что делать? Сначала добавляем проверку при регистрации, а затем запускаем следующий запрос для выявления дублей:

```
1 SELECT email, COUNT(1)
2   FROM customers
3  GROUP BY email
4 HAVING COUNT(1) > 1;
```

Мы группируем строки по e-mail и оставляем только те, у которых количество больше единицы. Так вы получите список e-mail с дублями. Что делать с ними дальше, решать вам: можно объединить или удалить лишние записи.

## Потерянные данные

Следующий паттерн разберем на примере. Представим, что у нас есть корзина пользователя, но кто-то случайно удалил ее. Мы хотим восстановить корзину из таблицы логов действий пользователей. Пример структуры такой таблицы:

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

| date       | user_id | item_id | in_out |
|------------|---------|---------|--------|
| 2024-03-02 | 1       | 2       | in     |
| 2024-03-03 | 1       | 1       | in     |
| 2024-03-05 | 1       | 1       | out    |
| 2024-03-05 | 1       | 2       | out    |
| 2024-03-08 | 1       | 4       | in     |
| 2024-03-09 | 1       | 3       | in     |
| 2024-03-11 | 1       | 2       | in     |

У нас есть дата, `user_id`, `item_id`, и действие `in_out`, обозначающее добавление или удаление товара из корзины. В итоге мы хотим получить следующую таблицу:

| user_id | item_id | in_out |
|---------|---------|--------|
| 1       | 2       | out    |
| 1       | 1       | in     |
| 1       | 4       | in     |
| 1       | 3       | in     |

Для каждого товара нас интересует последнее действие. Например, товар 2 был удален из корзины, а товары 1, 4 и 3 остались. Как это можно сделать проще всего? Есть несколько способов, включая использование аналитических функций, но наиболее простой подход следующий:

```

1 SELECT user_id, item_id,
2         SUBSTR(MAX(CONCAT(date::text, in_out)), 10)
3 FROM basket
4 GROUP BY user_id, item_id;
```

Мы берем корзину, группируем ее по `user_id` и `item_id`. Затем объединяем дату и поле `in_out` в текстовую строку. После этого ищем максимум среди этих значений. Очевидно, максимум определяется по дате. Когда мы нашли

максимум, `SUBSTR` берет с десятого символа значение `in_out`, то есть нам не нужно искать, например, первую или последнюю запись, — мы просто сравниваем конкатенированные строки и берем максимальное значение. Так мы получаем информацию о последнем действии для каждого товара в корзине.

## Вопросы на собеседовании

### В каком порядке выполняются инструкции в SQL?

Внутри СУБД операторы в запросе выполняются в определенном порядке. Это происходит в следующем порядке:

1. **FROM**: движок СУБД сначала определяет, из каких таблиц будут извлекаться данные.
2. **ON**: затем обрабатываются условия соединения, заданные в части `ON`, где происходит фильтрация до выполнения самого соединения.
3. **JOIN**: выполняется операция соединения таблиц, объединяющая данные на основе указанных условий.
4. **WHERE**: на этом этапе происходит фильтрация строк, удовлетворяющих условиям запроса.
5. **GROUP BY**: строки группируются по заданным столбцам, чтобы впоследствии можно было применять агрегатные функции.
6. **HAVING**: применяется фильтрация к агрегированным данным, отсекаются группы, не удовлетворяющие условиям.
7. **SELECT**: извлекаются конкретные столбцы, указанные в запросе, для формирования итогового набора данных, при необходимости выполняются операции с данными.
8. **DISTINCT**: убираются дубликаты из результатов, если это необходимо.
9. **ORDER BY**: сортировка данных по указанным столбцам в нужном порядке.
10. **LIMIT/OFFSET**: окончательно определяются строки для вывода, ограничивается количество возвращаемых результатов и осуществляется выборка с указанием смещения.

Эта последовательность объясняет, как движок СУБД обрабатывает запросы. Знание порядка выполнения инструкций позволяет оптимально структурировать запросы и эффективно обрабатывать данные.

Для упрощения понимания последовательности выполнения операторов в SQL-запросе представим, что мы работаем с двумя большими стопками документов. Сначала мы фильтруем и убираем ненужные страницы, затем объединяем их, группируем по определённым признакам, отмечаем важные элементы, и в итоге упорядочиваем и сокращаем по необходимости.

## В чем отличие между базой данных и системой управления базами данных (СУБД)?

База данных и система управления базами данных (СУБД) — это два взаимосвязанных, но различных понятия:

- **База данных** — это организованная коллекция данных, которая хранится и управляется в цифровом формате. Это сами данные, с которыми ведётся работа.
- **СУБД** — это программное обеспечение, которое управляет базой данных и обеспечивает доступ к ней. СУБД предоставляет интерфейс для выполнения операций над данными, таких как создание, чтение, обновление и удаление. Она также заботится о таких аспектах, как целостность данных, управление транзакциями, обеспечение безопасности, управление доступом и выполнение запросов.



Примером СУБД является PostgreSQL, с которой мы уже работали. СУБД освобождает пользователя от необходимости заниматься низкоуровневыми задачами по управлению данными, позволяя сосредоточиться на их анализе и обработке.

## Как реализуется связь «многие ко многим» в SQL?

Связь «многие ко многим» между двумя таблицами в реляционной базе данных реализуется с помощью третьей таблицы, называемой **таблицей связей** или

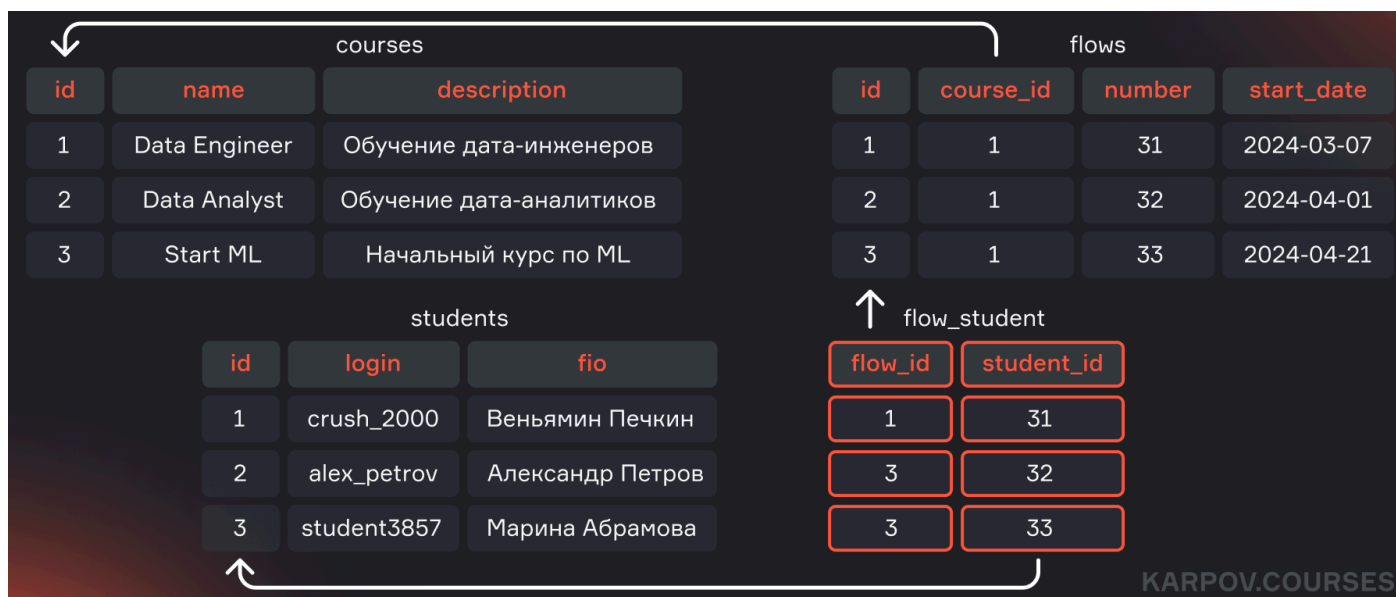
## соединительной таблицей.

Вот как выглядят различные типы связей:

- **Один к одному:** каждая запись в одной таблице соответствует одной записи в другой таблице. В большинстве случаев такие таблицы можно объединить в одну без изменения количества строк.
- **Один ко многим:** каждая запись в родительской таблице может иметь множество связанных записей в дочерней таблице. Это классическая связь между таблицами, где, например, один заказчик может иметь несколько заказов.
- **Многие ко многим:** любая запись из одной таблицы может соответствовать множеству записей в другой таблице и наоборот. Для реализации этой связи добавляется третья таблица, которая содержит внешние ключи обеих таблиц, связывая их друг с другом.

Пример:

Предположим, у нас есть таблицы `students` (студенты), `courses` (курсы) и `flows` (потоки). Каждый студент может записаться на несколько курсов и состоять в разных потоках. Для реализации связи «многие ко многим» создаётся таблица `flow_student`, которая содержит столбцы `student_id` и `flow_id`, представляющие собой внешние ключи, связывающие студентов и потоки.



Таким образом, связь «многие ко многим» организуется с помощью промежуточной таблицы, хранящей все возможные комбинации записей между двумя основными таблицами.

## Зачем нужны алиасы?

Алиасы (или псевдонимы) используются для временного присвоения имен таблицам или столбцам в SQL-запросах для упрощения работы с данными и улучшения читаемости запросов. Алиасы бывают двух типов:

1. **Алиасы для столбцов** — позволяют переименовать столбцы в результирующем наборе данных, что облегчает восприятие и понимание результатов запроса. Например, вместо вывода столбца с именем `total_sales` вы можете использовать более понятное имя `Total Sales`.

```
1 SELECT total_sales AS "Total Sales"
2 FROM Sales;
```

2. **Алиасы для таблиц** — используются для сокращения имен таблиц, особенно в запросах с соединениями, где может быть несколько таблиц с одинаковыми именами столбцов. Алиасы облегчают написание запросов и понимание того, из какой таблицы берутся данные.

```
1 SELECT s.name, c.course_name
2 FROM Students AS s
3 JOIN StudentCourses AS sc ON s.student_id = sc.student_id
4 JOIN Courses AS c ON sc.course_id = c.course_id;
```

Использование алиасов в запросах повышает их читаемость и ясность, особенно при работе с большими и сложными наборами данных.

## Как найти вторую по величине зарплату в таблице?

Для поиска второй по величине зарплаты в таблице можно использовать комбинацию операторов `ORDER BY`, `OFFSET` и `LIMIT`. Пример SQL-запроса:

```
1 SELECT salary
2 FROM Employees
3 ORDER BY salary DESC
4 OFFSET 1
5 LIMIT 1;
```

Этот запрос сортирует зарплаты в порядке убывания и пропускает первую строку ( `OFFSET 1` ), возвращая затем одну строку ( `LIMIT 1` ), то есть вторую по величине зарплату.



Однако важно уточнить у заказчика, как обрабатывать случаи, когда несколько сотрудников имеют одинаковую самую высокую или вторую по величине зарплату. Если таких сотрудников несколько, возможно, потребуется использовать аналитические функции, такие как `DENSE_RANK()` или `ROW_NUMBER()`, чтобы более точно идентифицировать вторую по величине зарплату:

```
1 SELECT DISTINCT salary
2 FROM (
3     SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rank
4     FROM Employees
5 ) ranked_salaries
6 WHERE rank = 2;
```

Этот запрос использует функцию `DENSE_RANK()`, которая присваивает ранги зарплатам, и выбирает те, которые имеют второй ранг.

## Чем отличается `JOIN` от `UNION` ?

`JOIN` и `UNION` — это две различные операции, используемые в SQL для работы с несколькими таблицами, однако их применение и результаты отличаются.



- `JOIN`: объединяет строки из двух или более таблиц на основе логического условия, определенного через оператор `ON`. `JOIN` работает с горизонтальным объединением данных, обогащая строки из одной таблицы информацией из другой.

- **Пример использования JOIN** : объединение таблиц `Employees` и `Departments` для получения информации о сотрудниках и их отделах:

```
1 SELECT e.name, d.department_name
2 FROM Employees e
3 JOIN Departments d ON e.department_id = d.id;
```

В этом примере каждая строка в таблице `Employees` объединяется с соответствующей строкой из таблицы `Departments` на основе идентификатора отдела ( `department_id` ).

- **UNION** : объединяет результаты двух или более SQL-запросов в один результирующий набор. **UNION** работает с вертикальным объединением данных, складывая строки из разных запросов. При этом автоматически удаляются дубликаты строк, если они есть. Для сохранения всех строк, включая дубликаты, используется **UNION ALL** .

- **Пример использования UNION** : объединение списков клиентов из двух разных таблиц `Customers_2023` и `Customers_2024` :

```
1 SELECT customer_name, email FROM Customers_2023
2 UNION
3 SELECT customer_name, email FROM Customers_2024;
```

В этом случае данные из обоих запросов объединяются в один набор результатов, исключая дублирующиеся строки. **UNION** требует, чтобы количество и порядок столбцов в объединяемых запросах совпадали.

## Какие виды JOIN кроме OUTER и INNER вы знаете?

В SQL помимо **OUTER** и **INNER** существует несколько видов операций соединения (**JOIN**), каждый из которых имеет свое предназначение и особенности:

- **CROSS JOIN** : создает декартово произведение двух таблиц, то есть каждая строка из первой таблицы соединяется с каждой строкой из второй таблицы. Результат содержит все возможные комбинации строк.

```
1 SELECT e.name, d.department_name
2 FROM Employees e
```

```
3 CROSS JOIN Departments d;
```

- **SELF JOIN** : это соединение таблицы с самой собой. Используется, например, для нахождения отношений между записями в одной таблице, таких как иерархии.

```
1 SELECT e1.name AS Employee, e2.name AS Manager
2 FROM Employees e1
3 JOIN Employees e2 ON e1.manager_id = e2.id;
```

- **Anti-Join**: этот тип соединения позволяет найти строки в одной таблице, для которых нет соответствующих строк в другой таблице. В SQL Anti-Join часто реализуется с использованием **LEFT JOIN** в комбинации с **WHERE** для отбора **NULL** значений.

```
1 SELECT Employees.name
2 FROM Employees
3 LEFT JOIN Departments ON Employees.department_id = Departments.id
4 WHERE Departments.id IS NULL;
```

Антиджоин официально не выделяют в качестве одного из типов соединений, однако это понятие бывает используется и следует его знать.

## Чем **COUNT(1)** отличается от **COUNT(\*)** и **COUNT(Name)** ?

В SQL функции **COUNT()** используются для подсчета количества строк или значений в наборе данных, но они имеют различные особенности в зависимости от параметров:

- **COUNT(\*)** : подсчитывает количество строк в результирующем наборе, включая строки с **NULL** значениями. При использовании **COUNT(\*)** движок обрабатывает все столбцы в таблице, смотря их значения.

```
1 SELECT COUNT(*) FROM Employees;
```

- **COUNT(1)** : работает аналогично **COUNT(\*)** , подсчитывая количество строк в таблице. Однако движок СУБД уже проверять все столбцы, поэтому использование константы в качестве аргумента как правило приводит уменьшению времени выполнения запроса.

```
1 SELECT COUNT(1) FROM Employees;
```

- **COUNT(Name)** : подсчитывает количество непустых ( **NOT NULL** ) значений в указанном столбце **Name** . Это полезно, когда необходимо узнать количество заполненных записей в конкретном столбце.

```
1 SELECT COUNT(Name) FROM Employees;
```

Таким образом, **COUNT(\*)** и **COUNT(1)** подсчитывают все строки, а **COUNT(Name)** подсчитывает только строки с непустыми значениями в столбце **Name** .

## Чем отличается **WHERE** от **HAVING** ?

**WHERE** и **HAVING** — это операторы SQL, которые используются для фильтрации данных, но применяются на разных этапах обработки запроса:

- **WHERE** : используется для фильтрации строк до выполнения группировки данных и применения агрегатных функций. **WHERE** задает условия, которым должны удовлетворять строки из исходного набора данных.

```
1 SELECT * FROM Employees
2 WHERE department_id = 1;
```

- **HAVING** : применяется для фильтрации данных после выполнения группировки, то есть для отбора групп, удовлетворяющих определенным условиям. Используется в сочетании с **GROUP BY** .

```
1 SELECT department_id, COUNT(*) as num_employees
2 FROM Employees
3 GROUP BY department_id
```

```
4 HAVING COUNT(*) > 10;
```

Оба оператора могут быть использованы одновременно в одном запросе, если необходимо фильтровать данные как до, так и после группировки:

```
1 SELECT department_id, COUNT(*) as num_employees
2 FROM Employees
3 WHERE salary > 50000
4 GROUP BY department_id
5 HAVING COUNT(*) > 10;
```

Обратите внимание, что фильтрацию данных по неагрегированным значениям можно делать как в блоке WHERE, так и в блоке HAVING. Например, результат следующих запросов будет одинаковый:

```
1 SELECT sex, COUNT(user_id)
2 FROM users
3 WHERE sex != 'male'
4 GROUP BY sex
```

```
1 SELECT sex, COUNT(user_id)
2 FROM users
3 GROUP BY sex
4 HAVING sex != 'male'
```

Однако делать фильтрацию по неагрегированным данным рекомендуется именно в блоке WHERE, т.е. заранее. В таком случае ненужные данные убираются из расчётов ещё до группировки и не расходуются вычислительные ресурсы на подсчёт значений, которые всё равно будут отфильтрованы вами позже. Это важный момент, касающийся оптимизации SQL-запросов.

## Что такое подзапрос и когда его следует использовать?

**Подзапрос** — это запрос, который выполняется внутри другого запроса. Он используется для работы с данными, которые были получены или вычислены в другом запросе. Подзапросы могут быть полезны в ситуациях, когда необходимо выполнить дополнительную обработку или фильтрацию данных, которые уже были извлечены или агрегированы. Хотя большинство подзапросов можно заменить на объединения (join), иногда это приводит к

таким сложным и запутанным конструкциям, что их становится трудно читать и понимать.

Пример — получение имён сотрудников с самой высокой зарплатой:

```
1 SELECT name
2 FROM Employees
3 WHERE salary = (SELECT MAX(salary) FROM Employees);
```

Подзапросы являются мощным инструментом, так как они позволяют легко использовать результаты одного запроса в другом, и их можно применять практически в любой части SQL-запроса.

## Чем отличаются агрегатные, аналитические и оконные функции?

**Агрегатные функции** служат для вычисления агрегированных значений, таких как сумма, количество, среднее значение или максимум. Для этого данные группируются по одному или нескольким полям, и затем над каждой группой выполняется вычисление агрегата. Примером может быть нахождение общей суммы продаж по каждому региону.

**Аналитические и оконные функции** — это на самом деле один и тот же тип функций в SQL, и между ними нет различий. Эти функции позволяют выполнять сложные вычисления и добавлять результаты к каждой строке в наборе данных без потери информации о самих строках. В отличие от агрегатных функций, которые сокращают число строк в результате запроса, аналитические и оконные функции возвращают все строки исходной таблицы, добавляя к каждой строке результаты вычислений. Например, они могут быть использованы для расчета скользящего среднего, ранжирования или кумулятивной суммы.

## Чем отличаются DELETE и TRUNCATE?

Операции **DELETE** и **TRUNCATE** используются для удаления данных из таблиц, но функционируют они по-разному.

- **DELETE** удаляет строки из таблицы построчно и обычно используется с условиями фильтрации для удаления определённых записей. Вы можете удалить все записи из таблицы с помощью **DELETE**, но этот процесс будет медленнее, поскольку каждая строка удаляется отдельно, что требует больше времени и ресурсов.

- **TRUNCATE**, с другой стороны, очищает всю таблицу целиком и сразу, фактически «выбрасывая» все её содержимое. Это более быстрый и эффективный метод удаления всех данных из таблицы, так как не происходит построчного удаления. **TRUNCATE** следует использовать, когда необходимо полностью очистить таблицу от данных без сохранения информации о конкретных удалённых строках.

На этом теоретическая часть подошла к концу. Нужно помнить, что SQL, как и другие инструменты дата-инженера, требует практики. Поэтому настоятельно рекомендуем к прохождению наш бесплатный [симулятор SQL](#), который позволит повторить пройденный материал и закрепить знания. Пусть освоенные в этом блоке навыки помогут успешно пройти любые собеседования и придадут уверенности на новом рабочем месте.

## Резюме

1. **Представление** — это таблица, которая не хранит данные, а извлекает их из других таблиц при необходимости. Таким образом, при использовании представления не происходит экономии вычислительных ресурсов. Представления используются для удобства и улучшения читаемости запросов.
2. **Материализованные представления** выглядят как обычные, но результаты запроса сохраняются в физическую таблицу (которая хранится на диске). Они поддерживаются не во всех СУБД, и работа с ними может существенно различаться.
3. **Оптимизация запросов** позволяет улучшить производительность систем, оптимизировать их совместное использование, а также сэкономить вычислительные ресурсы.

### Основные способы оптимизации запросов:

- **Индексы** — структуры данных, которые используются для ускорения поиска и сортировки данных в таблицах. Они создаются на одном или нескольких столбцах таблицы и позволяют быстро находить строки без необходимости сканировать всю таблицу.
- **Анализ плана запроса** — позволяет понять, как выполняется запрос под капотом СУБД и ориентировочные затрачиваемые ресурсы (стоимость запроса).

4. **Регулярные выражения** позволяют находить строки, соответствующие заданным условиям.
5. **Рекурсивные запросы** в SQL позволяют собирать данные в цикле. Они полезны для построения иерархий, таких как организационные структуры или получение упорядоченных списков (например, всех дат за период).